PREFIX SUM & SUFFIX SUM
ULTIMATE INTERVIEW HANDBOOK

1D Arrays • 2D Matrices • HashMap Patterns • Difference Arrays • Prefix XOR • Advanced Problems

Concept recognition • Mathematical derivations • Formula library • Complete Java implementations • Dry runs • Complexity • Edge cases • Interview problem roadmap

Java | IntelliJ IDEA | Beginner to Advanced

A practical guide to choosing the right technique, not just memorizing code

How to use this handbook

• Every code block is a standalone Java class. Save it using the same name as its public class.

• Start with the recognition guide, then learn the invariant and formula, dry-run the example, and finally code without looking.

• Indices are 0-based. Unless stated otherwise, ranges [L,R] are inclusive.

• Use long for cumulative sums and answers unless constraints prove int is safe.

• Prefix sum is usually for static data. For frequent updates mixed with queries, learn Fenwick Tree or Segment Tree.

• This handbook focuses on prefix/suffix and related cumulative techniques. Sliding window is a separate pattern and should be studied as its own topic.

Standard import: `import java.util.*;`

1. Recognition: how do I know which algorithm to use?

Question wording / clue	Technique	Why it fits
Many sum queries [L,R] on unchanged array	1D prefix sum	Precompute cumulative totals; query by subtraction.
Need sum from i to end / right-side totals	Suffix sum	Precompute from right to left.
Need left and right contribution at each index	Prefix + suffix / running total	Avoid rescanning both sides.
Count subarrays with sum K; negatives possible	Prefix + HashMap frequency	A previous prefix must equal current-K.
Longest subarray with sum K; negatives possible	Prefix + first-index HashMap	Earliest occurrence maximizes length.
Any zero-sum subarray / count zero-sum subarrays	Prefix + HashMap	Repeated prefix values imply zero-sum interval.
Subarray sum divisible by K	Prefix remainder frequency	Equal remainders have divisible difference.
Many interval additions, final array needed	Difference array	Two boundary changes per update.
Many rectangle sums in static matrix	2D prefix sum	Four-corner inclusion-exclusion.
Many rectangle additions, final matrix needed	2D difference array	Four corner changes per update.
Subarray XOR equals K	Prefix XOR + HashMap	Previous XOR = current XOR K.
Point updates + range sums	Fenwick / Segment Tree	Static prefix becomes stale after updates.
Max-sum rectangle in matrix	Compress rows/columns + Kadane	Reduce 2D search to 1D maximum subarray.

Recognition checklist

1. Is the required segment contiguous? Prefix identities often apply to contiguous ranges/subarrays.

2. Are there repeated range queries on unchanged data? Precompute prefix sums.

3. Can values be negative? For target-sum subarrays, prefer prefix + HashMap over a positivity-based sliding window.

4. Are updates applied to ranges before a final output? Difference array.

5. Is it a rectangle in a matrix? 2D prefix sum or 2D difference array, depending on query vs update.

6. Are updates interleaved with queries? Consider Fenwick Tree or Segment Tree.

Important: Prefix sum, suffix sum, difference array and sliding window solve different operation patterns. Do not substitute one for another just because the statement mentions a subarray.

2. Prefix sum 1D: definition and both conventions

Prefix sum stores cumulative totals from the start. It removes repeated addition over overlapping ranges.

```
a =       [ 2, 4, 1, 7, 3 ]
index       0  1  2  3  4

Inclusive prefix:
P =       [ 2, 6, 7, 14, 17 ]

Recommended padded prefix (n+1):
pref =    [ 0, 2, 6, 7, 14, 17 ]
index       0  1  2  3   4   5
```

Recommended formulas

Build: pref[0] = 0; pref[i+1] = pref[i] + a[i].

Inclusive range [L,R]: sum(L,R) = pref[R+1] - pref[L].

Prefix through i: pref[i+1]. **Sum before i:** pref[i]. **Sum after i:** pref[n] - pref[i+1].

2.1 Build and query prefix sums

Use the n+1 convention consistently. It handles ranges beginning at index zero without a special case.

```
import java.util.*;

public class PrefixSum1D {
    static long[] build(int[] a) {
        long[] pref = new long[a.length + 1];
        for (int i = 0; i < a.length; i++)
            pref[i + 1] = pref[i] + a[i];
        return pref;
    }

    static long query(long[] pref, int left, int right) {
        int n = pref.length - 1;
        if (left < 0 || right >= n || left > right)
            throw new IllegalArgumentException("Invalid range");
        return pref[right + 1] - pref[left];
    }

    public static void main(String[] args) {
        int[] a = {2, 4, 1, 7, 3};
        long[] p = build(a);
        System.out.println(Arrays.toString(p));
        System.out.println(query(p, 1, 3)); // 12
        System.out.println(query(p, 0, 4)); // 17
    }
}
```

Complexity: Build O(n), each query O(1), total O(n+q), space O(n).

Dry run

Query	Calculation	Result
sum(0,2)	pref[3]-pref[0] = 7-0	7
sum(1,3)	pref[4]-pref[1] = 14-2	12
sum(2,4)	pref[5]-pref[2] = 17-6	11
sum(0,4)	pref[5]-pref[0] = 17-0	17

Brute force can cost O(n) per query. Prefix preprocessing costs O(n), then q queries cost O(q): total O(n+q).

3. Suffix sum 1D

Suffix sum stores the total from index i through the last element. Here suffix[i] includes a[i].

```
a       = [2, 4, 1, 7, 3]
suffix  = [17,15,11,10,3,0]
                        ^ padded suffix[n] = 0
```

Build: suffix[n] = 0; suffix[i] = suffix[i+1] + a[i].

Inclusive range [L,R]: sum(L,R) = suffix[L] - suffix[R+1].

3.1 Build suffix and query

Iterate from right to left. The extra final zero makes the formula consistent.

```
import java.util.*;

public class SuffixSum1D {
    static long[] build(int[] a) {
        long[] s = new long[a.length + 1];
        for (int i = a.length - 1; i >= 0; i--)
            s[i] = s[i + 1] + a[i];
        return s;
    }

    static long query(long[] s, int left, int right) {
        int n = s.length - 1;
        if (left < 0 || right >= n || left > right)
            throw new IllegalArgumentException("Invalid range");
        return s[left] - s[right + 1];
    }

    public static void main(String[] args) {
        int[] a = {2, 4, 1, 7, 3};
        long[] s = build(a);
        System.out.println(Arrays.toString(s)); // [17,15,11,10,3,0]
        System.out.println(query(s, 1, 3)); // 12
    }
}
```

Complexity: Build O(n), query O(1), space O(n).

When suffix sum is useful

• Right-side totals at each index.

• Pivot / equilibrium checks.

• Combining left and right contributions in counting or dynamic programming.

• Computing suffix totals for many queries.

• Product-except-self and other left/right contribution problems use the same precomputation idea, even though the operation is multiplication.

4. Prefix and suffix together: left/right patterns

4.1 Left sum and right sum at every index

At i, left excludes a[i], and right excludes a[i]. Derive right from total to use O(1) extra space.

```
public class LeftRightSum {
    static void show(int[] a) {
        long total = 0;
        for (int x : a) total += x;

        long left = 0;
        for (int i = 0; i < a.length; i++) {
            long right = total - left - a[i];
            System.out.println(i + ": left=" + left + ", right=" + right);
            left += a[i];
        }
    }

    public static void main(String[] args) {
        show(new int[]{1, 7, 3, 6, 5, 6});
    }
}
```

Complexity: O(n) time, O(1) extra space.

4.2 Pivot / equilibrium index

Find an index where the sum strictly to its left equals the sum strictly to its right.

```
public class PivotIndex {
    static int find(int[] a) {
        long total = 0;
        for (int x : a) total += x;

        long left = 0;
        for (int i = 0; i < a.length; i++) {
            long right = total - left - a[i];
            if (left == right) return i;
            left += a[i];
        }
        return -1;
    }

    public static void main(String[] args) {
        System.out.println(find(new int[]{1,7,3,6,5,6})); // 3
    }
}
```

Complexity: O(n) time, O(1) extra space.

4.3 Product of array except self

Same left/right idea with products: answer[i] = product strictly left of i × product strictly right of i. No division, so zeros are handled.

```
import java.util.*;

public class ProductExceptSelf {
    static long[] solve(int[] a) {
        int n = a.length;
        long[] ans = new long[n];
        long leftProduct = 1;

        for (int i = 0; i < n; i++) {
            ans[i] = leftProduct;
            leftProduct *= a[i];
        }

        long rightProduct = 1;
        for (int i = n - 1; i >= 0; i--) {
            ans[i] *= rightProduct;
            rightProduct *= a[i];
        }
        return ans;
    }

    public static void main(String[] args) {
```

```
        System.out.println(Arrays.toString(solve(new int[]{1,2,3,4})));
        // [24,12,8,6]
    }
}
```

Complexity: O(n) time, O(1) extra space excluding output. Use BigInteger if products can exceed long.

5. Prefix sum + HashMap: subarray identity

This is a core interview pattern, especially when negative values are allowed.

```
Let P[i] = sum(a[0..i]).
Sum of subarray (j+1..i) = P[i] - P[j].

For target K:
P[i] - P[j] = K
P[j] = P[i] - K

At current prefix P, look up earlier prefix P-K.
```

Frequency map: counts how many earlier prefixes have each sum. Initialize 0 -> 1 for the empty prefix. Count matches before inserting current prefix to avoid counting an empty subarray for K=0.

5.1 Count subarrays with sum K

Works with positive, zero and negative numbers. HashMap stores prefix-sum frequencies.

```
import java.util.*;

public class SubarraySumK {
    static long count(int[] a, long k) {
        Map<Long, Long> freq = new HashMap<>();
        freq.put(0L, 1L);

        long prefix = 0, answer = 0;
        for (int x : a) {
            prefix += x;
            answer += freq.getOrDefault(prefix - k, 0L);
            freq.put(prefix, freq.getOrDefault(prefix, 0L) + 1L);
        }
        return answer;
    }

    public static void main(String[] args) {
        System.out.println(count(new int[]{1,2,3,-2,2}, 3)); // 4
    }
}
```

Complexity: Expected O(n) time, O(n) space.

Dry run: [1,2,3], K=3

Read	Prefix	Need prefix-K	Previous matches	Answer
1	1	-2	0	0
2	3	0	1	1
3	6	3	1	2

Matching segments are [1,2] and [3], so count = 2.

5.2 Longest subarray with sum K

Store the earliest index for each prefix. For current prefix P at i, previous P-K at j gives length i-j. Keep the first occurrence only.

```
import java.util.*;

public class LongestSubarrayK {
    static int longest(int[] a, long k) {
        Map<Long, Integer> first = new HashMap<>();
        first.put(0L, -1);

        long prefix = 0;
        int best = 0;
        for (int i = 0; i < a.length; i++) {
            prefix += a[i];

            Integer j = first.get(prefix - k);
            if (j != null) best = Math.max(best, i - j);

            first.putIfAbsent(prefix, i);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longest(new int[]{10,5,2,7,1,9}, 15)); // 4
    }
}
```

Complexity: Expected O(n) time, O(n) space.

5.3 Does a zero-sum subarray exist?

If a prefix sum repeats, the elements between those two prefix positions sum to zero.

```
import java.util.*;

public class ZeroSumExists {
    static boolean exists(int[] a) {
        Set<Long> seen = new HashSet<>();
        seen.add(0L);
        long prefix = 0;

        for (int x : a) {
            prefix += x;
            if (seen.contains(prefix)) return true;
            seen.add(prefix);
        }
        return false;
    }

    public static void main(String[] args) {
        System.out.println(exists(new int[]{4, 2, -3, 1, 6})); // true
    }
}
```

Complexity: Expected O(n) time, O(n) space.

5.4 Count zero-sum subarrays

Use the frequency of the current prefix itself. Each earlier identical prefix forms one zero-sum subarray.

```
import java.util.*;

public class CountZeroSum {
    static long count(int[] a) {
        Map<Long, Long> freq = new HashMap<>();
        freq.put(0L, 1L);
        long prefix = 0, ans = 0;

        for (int x : a) {
            prefix += x;
            ans += freq.getOrDefault(prefix, 0L);
            freq.put(prefix, freq.getOrDefault(prefix, 0L) + 1L);
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(count(new int[]{0, 0, 0})); // 6
    }
}
```

Complexity: Expected O(n) time, O(n) space.

6. Prefix remainders and transformed arrays

6.1 Count subarrays whose sum is divisible by K

If two prefix sums have the same remainder modulo K, their difference is divisible by K. Normalize remainders with floorMod, especially for negative sums.

```
import java.util.*;

public class SubarraysDivisibleByK {
    static long count(int[] a, int k) {
        if (k == 0) throw new IllegalArgumentException("k != 0");
        long mod = Math.abs((long) k);
        Map<Long, Long> freq = new HashMap<>();
        freq.put(0L, 1L);

        long prefix = 0, ans = 0;
        for (int x : a) {
            prefix += x;
            long rem = Math.floorMod(prefix, mod);
            ans += freq.getOrDefault(rem, 0L);
            freq.put(rem, freq.getOrDefault(rem, 0L) + 1L);
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(count(new int[]{4,5,0,-2,-3,1}, 5)); // 7
    }
}
```

Complexity: Expected O(n) time, O(n) space.

6.2 Longest subarray with sum divisible by K

Same remainder means the segment between the two prefix positions is divisible by K. Keep the earliest index for each remainder.

```
import java.util.*;

public class LongestDivisibleSubarray {
    static int longest(int[] a, int k) {
        if (k == 0) throw new IllegalArgumentException("k != 0");
        long mod = Math.abs((long) k);
        Map<Long, Integer> first = new HashMap<>();
        first.put(0L, -1);

        long prefix = 0;
        int best = 0;
        for (int i = 0; i < a.length; i++) {
            prefix += a[i];
            long rem = Math.floorMod(prefix, mod);
            Integer j = first.get(rem);
            if (j != null) best = Math.max(best, i - j);
            first.putIfAbsent(rem, i);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longest(new int[]{2,7,6,1,4,5}, 3));
    }
}
```

Complexity: Expected O(n) time, O(n) space.

6.3 Equal number of 0s and 1s

Transform 0 to -1 and 1 to +1. Equal counts mean transformed sum is zero. Find the longest zero-sum subarray.

```
import java.util.*;

public class EqualZeroOne {
    static int longest(int[] a) {
        Map<Integer, Integer> first = new HashMap<>();
        first.put(0, -1);
        int sum = 0, best = 0;

        for (int i = 0; i < a.length; i++) {
            sum += (a[i] == 0 ? -1 : 1);
            Integer j = first.get(sum);
            if (j != null) best = Math.max(best, i - j);
            else first.put(sum, i);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longest(new int[]{0,1,0,1,1,0}));
    }
}
```

Complexity: O(n) time, O(n) space. Input must contain only 0 and 1.

6.4 Count binary subarrays with sum K

For a binary array, this prefix-frequency approach works for K=0 and positive K.

```
import java.util.*;

public class BinarySubarraysSum {
    static long count(int[] a, int goal) {
        Map<Integer, Long> freq = new HashMap<>();
        freq.put(0, 1L);
        int sum = 0;
        long ans = 0;

        for (int x : a) {
            sum += x;
            ans += freq.getOrDefault(sum - goal, 0L);
            freq.put(sum, freq.getOrDefault(sum, 0L) + 1L);
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(count(new int[]{1,0,1,0,1}, 2)); // 4
    }
}
```

Complexity: O(n) time, O(n) space.

7. Difference array: range updates

Use this when many operations add a value to intervals, and you need the final array after all operations. Do not confuse it with prefix sums for range queries.

```
Start: [0,0,0,0,0,0]
Add +5 to [1,3]:
diff[1] += 5
diff[4] -= 5

diff:  [0,5,0,0,-5,0]
Prefix reconstruction:
       [0,5,5,5,0,0]
```

Update [L,R] by v: diff[L] += v; diff[R+1] -= v. Reconstruct with a running sum.

7.1 Apply q range additions

Allocate n+1 difference entries so R+1 is always a valid boundary when R=n-1.

```
import java.util.*;

public class DifferenceArray {
    static long[] apply(int n, int[][] updates) {
        long[] diff = new long[n + 1];
        for (int[] u : updates) {
            int l = u[0], r = u[1], v = u[2];
            if (l < 0 || r >= n || l > r)
                throw new IllegalArgumentException("Bad range");
            diff[l] += v;
            diff[r + 1] -= v;
        }

        long[] ans = new long[n];
        long running = 0;
        for (int i = 0; i < n; i++) {
            running += diff[i];
            ans[i] = running;
        }
        return ans;
    }

    public static void main(String[] args) {
        int[][] u = {{1,3,5},{2,5,2},{0,2,1}};
        System.out.println(Arrays.toString(apply(6,u)));
        // [1,6,8,7,2,2]
    }
}
```

Complexity: O(q+n) time, O(n) space.

When to use / not use

• Use for offline range additions followed by final materialization.

• Use for interval booking, flight seat counts, coverage counts and range increments.

• Not enough alone for arbitrary range-sum queries after each update.

• For point updates + range sums, use Fenwick Tree or Segment Tree.

8. 2D prefix sum: rectangle queries

A 2D prefix matrix stores the sum of the rectangle from (0,0) through (r,c). It supports any axis-aligned rectangular sum in O(1).

```
A:
1 2 3
4 5 6
7 8 9

Padded pref:
0  0  0  0
0  1  3  6
0  5 12 21
0 12 27 45
```

Build: pref[r+1][c+1] = A[r][c] + pref[r][c+1] + pref[r+1][c] - pref[r][c].

Rectangle [r1..r2] × [c1..c2]: pref[r2+1][c2+1] - pref[r1][c2+1] - pref[r2+1][c1] + pref[r1][c1].

8.1 Build 2D prefix and answer rectangle query

Coordinates are inclusive and zero-based. Matrix must be rectangular.

```
import java.util.*;

public class PrefixSum2D {
    static long[][] build(int[][] a) {
        int rows = a.length;
```

```
        int cols = rows == 0 ? 0 : a[0].length;
        long[][] p = new long[rows + 1][cols + 1];

        for (int r = 0; r < rows; r++) {
            if (a[r].length != cols)
                throw new IllegalArgumentException("Ragged matrix");
            for (int c = 0; c < cols; c++) {
                p[r+1][c+1] = a[r][c] + p[r][c+1]
                            + p[r+1][c] - p[r][c];
            }
        }
        return p;
    }

    static long query(long[][] p, int r1, int c1, int r2, int c2) {
        int rows = p.length - 1, cols = p[0].length - 1;
        if (r1 < 0 || c1 < 0 || r2 >= rows || c2 >= cols
                || r1 > r2 || c1 > c2)
            throw new IllegalArgumentException("Bad rectangle");

        return p[r2+1][c2+1] - p[r1][c2+1]
             - p[r2+1][c1] + p[r1][c1];
    }

    public static void main(String[] args) {
        int[][] a = {{1,2,3},{4,5,6},{7,8,9}};
        long[][] p = build(a);
        System.out.println(query(p,1,1,2,2)); // 28
        System.out.println(query(p,0,0,2,2)); // 45
        System.out.println(query(p,0,1,1,2)); // 16
    }
}
```

Complexity: Build O(RC), each query O(1), space O(RC).

Why four terms?

• Add the large prefix rectangle ending at the query's bottom-right.

• Subtract the rectangle above the requested region.

• Subtract the rectangle to its left.

• Add the top-left overlap back because it was subtracted twice.

2D use cases

• Many submatrix sum queries.

• Count marked cells: convert qualifying cells to 1 and others to 0.

• Count zeros, ones, obstacles, mines, pixels or cells with a property in any rectangle.

• Fast rectangular totals in image processing and grid analytics.

9. 2D difference array: rectangle updates

For adding v to every cell in rectangle [r1..r2] × [c1..c2], mark four corners, then perform a 2D prefix accumulation.

```
D[r1][c1]       += v
D[r1][c2+1]     -= v
D[r2+1][c1]     -= v
D[r2+1][c2+1]   += v

Rebuild:
A[r][c] = D[r][c] + top + left - diagonal
```

9.1 Multiple rectangle additions

Use an extra row and column so boundary updates at r2+1 or c2+1 are valid.

```
import java.util.*;

public class DifferenceArray2D {
    static long[][] apply(int rows, int cols, int[][] updates) {
        long[][] d = new long[rows+1][cols+1];

        for (int[] u : updates) {
            int r1=u[0], c1=u[1], r2=u[2], c2=u[3], v=u[4];
            if (r1<0 || c1<0 || r2>=rows || c2>=cols || r1>r2 || c1>c2)
                throw new IllegalArgumentException("Bad rectangle");
            d[r1][c1] += v;
            d[r1][c2+1] -= v;
            d[r2+1][c1] -= v;
            d[r2+1][c2+1] += v;
        }

        long[][] a = new long[rows][cols];
        for (int r=0; r<rows; r++) {
```

```
            for (int c=0; c<cols; c++) {
                long top = r>0 ? a[r-1][c] : 0;
                long left = c>0 ? a[r][c-1] : 0;
                long diag = (r>0 && c>0) ? a[r-1][c-1] : 0;
                a[r][c] = d[r][c] + top + left - diag;
            }
        }
        return a;
    }

    public static void main(String[] args) {
        int[][] u = {{0,0,1,1,5},{1,1,2,2,3}};
        for (long[] row : apply(3,3,u))
            System.out.println(Arrays.toString(row));
        // [5,5,0]
        // [5,8,3]
        // [0,3,3]
    }
}
```

Complexity: O(q+RC) time, O(RC) space.

10. Advanced 2D problem: count submatrices with sum K

A direct four-loop approach is too slow for large matrices. Fix a pair of row boundaries, compress the cells between them into a 1D array of column sums, then count subarrays with sum K using prefix + HashMap. Choose the smaller dimension for the pair when possible.

10.1 Count submatrices whose sum equals K

This version is O(min(R,C)^2 × max(R,C)) expected time after orienting the smaller dimension as the paired dimension. It supports negative matrix values.

```
import java.util.*;

public class CountSubmatricesSumK {
    static long count(int[][] mat, long target) {
        int R = mat.length;
        if (R == 0) return 0;
        int C = mat[0].length;
        for (int[] row : mat)
            if (row.length != C) throw new IllegalArgumentException();

        // If columns are fewer, transpose so paired dimension is smaller.
        if (C > R) {
            int[][] t = new int[C][R];
            for (int r=0; r<R; r++)
                for (int c=0; c<C; c++) t[c][r] = mat[r][c];
            mat = t;
            int tmp = R; R = C; C = tmp;
        }

        long answer = 0;
        for (int top=0; top<R; top++) {
            long[] col = new long[C];
            for (int bottom=top; bottom<R; bottom++) {
                for (int c=0; c<C; c++) col[c] += mat[bottom][c];

                Map<Long, Long> freq = new HashMap<>();
                freq.put(0L, 1L);
                long prefix = 0;
                for (long x : col) {
                    prefix += x;
                    answer += freq.getOrDefault(prefix-target, 0L);
                    freq.put(prefix, freq.getOrDefault(prefix, 0L)+1L);
                }
            }
        }
        return answer;
    }

    public static void main(String[] args) {
        int[][] m = {{0,1,0},{1,1,1},{0,1,0}};
        System.out.println(count(m, 0)); // 4
    }
}
```

Complexity: O(min(R,C)^2 × max(R,C)) expected time, O(max(R,C)) auxiliary space.

Recognition: if the question asks for the number of rectangular submatrices with target sum, think: fix two boundaries → compress → 1D subarray sum K.

11. Prefix XOR

XOR prefix uses the same cancellation idea because x ^ x = 0. XOR of subarray (j+1..i) equals prefixXor[i] ^ prefixXor[j].

Target K: previousXor = currentXor ^ K. Initialize frequency 0 -> 1.

11.1 Count subarrays with XOR K

Works for integer values and uses a frequency map of previous prefix XOR values.

```
import java.util.*;

public class SubarrayXorK {
    static long count(int[] a, int k) {
        Map<Integer, Long> freq = new HashMap<>();
        freq.put(0, 1L);
        int xor = 0;
        long ans = 0;

        for (int x : a) {
            xor ^= x;
            ans += freq.getOrDefault(xor ^ k, 0L);
            freq.put(xor, freq.getOrDefault(xor, 0L)+1L);
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(count(new int[]{4,2,2,6,4}, 6)); // 4
    }
}
```

Complexity: Expected O(n) time, O(n) space.

12. Prefix min/max and other cumulative scans

The word prefix does not always mean sum. A prefix maximum stores the largest value seen so far; suffix minimum stores the smallest value to the right. These scans are common in arrays, rainwater, stock and dominance problems.

12.1 Prefix maximum and suffix maximum

Build leftMax[i] = max(a[0..i]) and rightMax[i] = max(a[i..n-1]).

```
import java.util.*;

public class PrefixSuffixMax {
    static void build(int[] a) {
        int n=a.length;
        if (n==0) return;
        int[] left=new int[n], right=new int[n];

        left[0]=a[0];
        for (int i=1;i<n;i++) left[i]=Math.max(left[i-1],a[i]);

        right[n-1]=a[n-1];
        for (int i=n-2;i>=0;i--) right[i]=Math.max(right[i+1],a[i]);

        System.out.println(Arrays.toString(left));
        System.out.println(Arrays.toString(right));
    }

    public static void main(String[] args) {
        build(new int[]{4,2,0,3,2,5});
        // left:  [4,4,4,4,4,5]
        // right: [5,5,5,5,5,5]
    }
}
```

Complexity: O(n) time, O(n) space.

12.2 Trapping rain water using prefix/suffix maxima

At each index, water height is min(max wall to left, max wall to right) - current height, clamped at zero.

```
public class TrappingRainWater {
    static long trap(int[] h) {
        int n=h.length;
        if (n==0) return 0;
        int[] left=new int[n], right=new int[n];

        left[0]=h[0];
        for (int i=1;i<n;i++) left[i]=Math.max(left[i-1],h[i]);

        right[n-1]=h[n-1];
        for (int i=n-2;i>=0;i--) right[i]=Math.max(right[i+1],h[i]);

        long water=0;
        for (int i=0;i<n;i++)
            water += Math.max(0,Math.min(left[i],right[i])-h[i]);
        return water;
    }

    public static void main(String[] args) {
        System.out.println(trap(new int[]{4,2,0,3,2,5})); // 9
    }
}
```

Complexity: O(n) time, O(n) space. A two-pointer alternative achieves O(1) auxiliary space.

12.3 Prefix weighted sum

Sometimes each element has a position-based weight. Store both prefix sum and prefix weighted sum to answer weighted range calculations quickly.

```
public class WeightedPrefix {
    static long weightedRange(int[] a, int l, int r) {
        long[] p = new long[a.length+1];
        long[] wp = new long[a.length+1];

        for (int i=0;i<a.length;i++) {
            p[i+1] = p[i] + a[i];
            wp[i+1] = wp[i] + (long)(i+1)*a[i];
        }

        long sum = p[r+1]-p[l];
        long weighted = wp[r+1]-wp[l];
        // Sum of (i-l+1)*a[i] over i=l..r:
        return weighted - (long)l*sum;
    }

    public static void main(String[] args) {
        int[] a={2,3,4};
        System.out.println(weightedRange(a,1,2)); // 11: 1*3 + 2*4
    }
}
```

Complexity: O(n) preprocessing, O(1) per weighted range query, O(n) space.

13. Prefix frequency / coordinate transformations

13.1 Count subarrays with equal 0, 1 and 2

For a subarray to contain equal counts of 0, 1 and 2, the pair (count1-count0, count2-count0) must be equal at two prefix positions. Store the earliest occurrence of each pair to count or maximize lengths.

Equal 0s, 1s and 2s — longest subarray

Use a small immutable key class because Java arrays do not compare by contents by default.

```
import java.util.*;

public class Equal012 {
    static class Key {
        final int a,b;
        Key(int a,int b){this.a=a;this.b=b;}
        public int hashCode(){return 31*a+b;}
        public boolean equals(Object o){
            if(!(o instanceof Key)) return false;
            Key k=(Key)o;
            return a==k.a && b==k.b;
        }
    }

    static int longest(int[] a) {
        Map<Key,Integer> first=new HashMap<>();
        first.put(new Key(0,0),-1);
        int z=0,o=0,t=0,best=0;

        for(int i=0;i<a.length;i++){
            if(a[i]==0) z++;
            else if(a[i]==1) o++;
            else if(a[i]==2) t++;
            else throw new IllegalArgumentException("Only 0,1,2");

            Key key=new Key(o-z,t-z);
            Integer j=first.get(key);
            if(j!=null) best=Math.max(best,i-j);
            else first.put(key,i);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longest(new int[]{0,1,2,0,1,2}));
    }
}
```

Complexity: O(n) expected time, O(n) space.

13.2 Prefix sums with coordinate compression

When prefix values are large or sparse and you need ordered counts (for example, count earlier prefix sums in a value interval), coordinate compression maps sorted distinct values to compact indices. Combine it with a Fenwick Tree. This is an advanced extension, not needed for ordinary range-sum queries.

14. Dynamic range queries: when prefix arrays are not enough

Operation mix	Use	Typical cost
Static array, many range sums	Prefix sum	Build O(n), query O(1)
Point update, range sum	Fenwick Tree	Update/query O(log n)
Point update, flexible range aggregate	Segment Tree	Update/query O(log n)
Range add, point query	Difference + Fenwick Tree	Update/query O(log n)
Range add, range sum	Two Fenwick Trees or lazy Segment Tree	O(log n)

14.1 Fenwick Tree (Binary Indexed Tree) — Java

A Fenwick Tree maintains prefix sums under point updates. It is a standard next step after static prefix sums.

```
lowbit(i) = i & -i
update: i += lowbit(i)
query:  i -= lowbit(i)
```

Fenwick Tree: point add and range sum

Internal indices are 1-based. Public array indices are 0-based.

```
public class FenwickTree {
    private final long[] bit;
    private final int n;

    FenwickTree(int n) {
        this.n=n;
        bit=new long[n+1];
    }

    void add(int index, long delta) {
        for(int i=index+1;i<=n;i+=i&-i)
            bit[i]+=delta;
    }

    long prefixSum(int index) {
        long sum=0;
        for(int i=index+1;i>0;i-=i&-i)
            sum+=bit[i];
        return sum;
    }

    long rangeSum(int left,int right) {
        if(left>right) return 0;
        return prefixSum(right)-(left==0?0:prefixSum(left-1));
    }

    public static void main(String[] args) {
        FenwickTree f=new FenwickTree(5);
        int[] a={2,4,1,7,3};
        for(int i=0;i<a.length;i++) f.add(i,a[i]);
        System.out.println(f.rangeSum(1,3)); // 12
        f.add(2,5); // a[2] += 5
        System.out.println(f.rangeSum(1,3)); // 17
    }
}
```

Complexity: Point add O(log n), prefix/range sum O(log n), space O(n).

15. Common mistakes and debugging checklist

• Off-by-one: with n+1 prefix, inclusive [L,R] is pref[R+1]-pref[L].

• Mixing definitions: write what pref[i] represents before coding.

• Missing empty prefix: initialize 0 -> 1 in count-subarray patterns.

• Wrong HashMap order: count matches before inserting the current prefix.

• Longest length bug: preserve earliest index with putIfAbsent.

• Negative remainder bug: use Math.floorMod(prefix, absK), not prefix % K directly.

• Overflow: use long for prefix sums, counts, matrix totals and products where constraints require it.

• 2D sign error: +bottom-right, -top strip, -left strip, +overlap.

• R+1 boundary: allocate n+1 for difference arrays.

• Empty input: handle n=0 before accessing a[0] or a[n-1].

• Static vs dynamic: changing an array value invalidates ordinary precomputed prefixes.

• Subarray means contiguous: a subsequence is not necessarily a subarray.

Test cases to run before submitting

• Empty array and one-element array.

• Query [0,n-1], [0,0], [n-1,n-1].

• All zeros, all positive, all negative, mixed signs.

• Repeated prefix values and K=0.

• Target K not present and target K equal to full-array sum.

• 2D 1×1, one row, one column, full matrix and single-cell rectangle.

• Range update ending at n-1 and rectangle update touching last row/column.

16. Complexity reference

Pattern	Preprocessing / scan	Query / update	Space
1D prefix	O(n)	Range sum O(1)	O(n)
1D suffix	O(n)	Suffix/range sum O(1)	O(n)
Prefix + HashMap	O(n) expected	Count/longest in one scan	O(n)
Difference array	O(q+n)	Each range update O(1)	O(n)
2D prefix	O(RC)	Rectangle sum O(1)	O(RC)
2D difference	O(q+RC)	Each rectangle update O(1)	O(RC)
Submatrix sum K	O(min² × max) expected	All counted in scan	O(max)
Prefix XOR + map	O(n) expected	Count in one scan	O(n)
Fenwick Tree	O(n log n) via adds	Add/query O(log n)	O(n)

17. Interview problem roadmap

Foundation

• Build prefix sum array and answer Q range queries.

• Build suffix sum array and answer suffix/range queries.

• Left sum / right sum at every index.

• Pivot Index / Equilibrium Point.

• Running prefix maximum and suffix maximum.

• Product of Array Except Self.

Core subarray HashMap

• Subarray Sum Equals K.

• Maximum Size Subarray Sum Equals K.

• Zero Sum Subarray — existence and count.

• Subarray Sums Divisible by K.

• Longest Subarray with Sum Divisible by K.

• Contiguous Array — equal 0s and 1s.

• Binary Subarrays With Sum.

• Count subarrays with equal 0, 1 and 2.

• Subarray XOR Equals K.

Difference arrays

• Range Addition.

• Corporate Flight Bookings.

• Car Pooling / capacity over intervals.

• Apply multiple range increments.

• 2D rectangle range additions.

2D prefix / advanced

• Range Sum Query 2D — Immutable.

• Matrix Block Sum.

• Count Submatrices With Sum K.

• Number of Submatrices That Sum to Target.

• Maximum Sum Rectangle in a 2D Matrix.

• Count ones / zeros / obstacles in arbitrary subrectangles.

• 2D Fenwick Tree for dynamic rectangle sums.

Advanced data structures

• Range Sum Query — Mutable (Fenwick Tree or Segment Tree).

• Count of Range Sum.

• Reverse Pairs (merge-sort counting / Fenwick alternatives).

• Range add and range sum using two Fenwick Trees.

• Segment Tree with lazy propagation.

18. Problem-solving walkthrough: choose the implementation

If the statement says…	Ask yourself…	Implementation direction
'For each query, sum L through R'	Is the array unchanged?	Build pref[n+1], answer pref[R+1]-pref[L].
'How many subarrays sum to K?'	Can numbers be negative?	Prefix running sum + frequency map; lookup sum-K.
'Longest subarray sum K'	Do I need the longest, not count?	Map prefix to earliest index; putIfAbsent.
'Sum divisible by K'	What makes a difference divisible?	Equal prefix remainders; normalize with floorMod.
'Add x to every index L..R many times'	Do I need intermediate answers?	Difference array, then one prefix reconstruction.
'Sum of rectangle many times'	Are matrix values fixed?	Build 2D prefix; use four terms.
'Add x to each cell in many rectangles'	Only final matrix needed?	2D difference; four corners then 2D accumulation.
'Update an element, then query a range'	Are updates interleaved?	Fenwick Tree or Segment Tree.

19. Formula sheet — revise before interviews

Concept	Formula
1D padded prefix	pref[0]=0; pref[i+1]=pref[i]+a[i]
Inclusive range sum	pref[R+1]-pref[L]
Padded suffix	suffix[n]=0; suffix[i]=suffix[i+1]+a[i]
Suffix range sum	suffix[L]-suffix[R+1]
Subarray sum K	previousPrefix=currentPrefix-K
Count target-sum subarrays	freq[0]=1; ans += freq[prefix-K]
Longest target-sum subarray	first.putIfAbsent(prefix,index)
Divisible by K	same normalized prefix remainder
Prefix XOR target K	previousXor=currentXor XOR K
1D range add	diff[L]+=v; diff[R+1]-=v
2D prefix build	P[r+1][c+1]=A+top+left-diagonal
2D rectangle sum	+bottomRight - top - left + overlap
2D rectangle update	+v, -v, -v, +v at four corners
Pivot index	left == total-left-a[i]
Product except self	leftProduct[i] × rightProduct[i]
Fenwick lowbit	i & -i

20. Mini assessment

1. For a=[3,-1,4,2], write the n+1 prefix array.

2. Using that array, calculate sum(1,3), inclusive.

3. If suffix[i] includes a[i], what is suffix[2]?

4. Why initialize the prefix frequency map with 0 -> 1?

5. Why count matching prefixes before inserting the current prefix?

6. Why is prefix + HashMap safer than a normal sum sliding window when negatives exist?

7. What structure handles point updates plus range-sum queries?

8. What are the four corner updates for a 2D rectangle addition?

Answer key

1. [0,3,2,6,8].

2. pref[4]-pref[1] = 8-3 = 5.

3. 4+2 = 6.

4. It represents the empty prefix and counts segments starting at index 0.

5. To avoid counting an empty subarray at the current position.

6. Negative values break the monotonic expand/shrink behavior required by the usual sum window.

7. Fenwick Tree or Segment Tree.

8. D[r1][c1]+=v; D[r1][c2+1]-=v; D[r2+1][c1]-=v; D[r2+1][c2+1]+=v.

Final interview principle: Do not start by typing code. Identify the operation, define the stored state, derive the formula, choose the data structure, handle boundaries, prove the invariant, and state time and space complexity.